

Unit - 3 Language Components

Topic : Indenting Requirements

- Python एक block structured language हैं, जिसमें indentation (indent) का उपयोग code blocks को define करने के लिए किया जाता हैं।
- Python में { } (curly braces) का उपयोग नहीं होता, उसकी जगह indentation का उपयोग किया जाता हैं।
- Indentation से Python interpreter को पता चलता हैं कि कौन सा statement किस block के अंदर आता हैं।
- यदि indentation सही नहीं होगा तो IndentationError प्राप्त होता हैं।

1. Indentation क्या होता हैं?

किसी भी statement को दाएँ side (left margin से अंदर की ओर) लिखना indentation कहलाता हैं।

Python में आमतौर पर 4 spaces को एक indentation level माना जाता हैं। हम tab key का भी उपयोग कर सकते हैं, लेकिन consistency जरूरी हैं।

Indentation Example

```
if True:
    print("Hello")
    print("Python")
```

4 spaces

2. Indenting के नियम (Rules)

- समान block के सभी statements का indentation level एक जैसा होना चाहिए।
- किसी new block के statements को पिछले line से अधिक indent करना होता हैं।
- Block समाप्त होने पर indentation कम (dedent) करना होता हैं।
- Mixing of spaces and tabs एक ही block में नहीं करना चाहिए।
- आमतौर पर 4 spaces per indentation level का उपयोग best practice माना जाता हैं।

3. Indentation का उपयोग

→ Python में indentation का उपयोग निम्न कार्यों के लिए होता हैं -

- ① Control Statements (if, else, elif, for, while, etc.)
- ② Functions और Class के अंदर code define करने के लिए
- ③ Loop और Conditional block को define करने के लिए
- ④ Code को readable और logically structured बनाने के लिए

Why Important ?

Indentation code की structure और logic को साफ और readable बनाता हैं।

<u>Correct Indentation Example</u>	<u>5. Incorrect Indentation Example</u>
<pre>x = 10 if x > 5: print("x is greater than 5") else: print("x is less than or equal to 5") for i in range(3): print(i) print("Inside loop")</pre>	<pre>x = 10 if x > 5: print("x is greater than 5") else: print("x is less than or equal to 5") for i in range(3): print(i) print("Inside loop")</pre> Same level पर write नहीं होना चाहिए यह Loop के अंदर होना चाहिए था

6. Common Indentation Errors

- ① IndentationError: unindent does not match any outer indentation level
- ② unexpected indent
- ③ inconsistent use of tabs and spaces in indentation

Best Practices

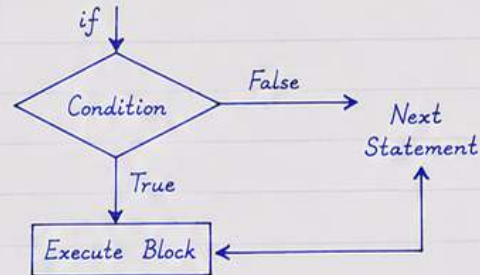
- ✓ हमेशा 4 spaces का उपयोग करें।
- ✓ एक ही code block में spaces और tabs को mix न करें।
- ✓ Code को properly align और indent करने की आदत डालें।

Topic : The if Statement

- if statement Python में decision making या conditional execution के लिए use किया जाता है।
- यह statement किसी condition को evaluate करता है और condition True होने पर block के statements को execute करता है।
- अगर condition False होती है तो if block skip कर दिया जाता है।

1. Basic Syntax

```
if condition:  
    statement(s)  
next statement
```



2. How it Works ?

- Python condition ko check करता है।
- यदि condition True होती है, तो उसी indentation level (block) के सभी statements execute होते हैं।
- यदि condition False होती है, तो if block को skip कर दिया जाता है। और control next statement पर चला जाता है।

Important :

Python indentation (spaces) के द्वारा block define करता है, इसलिए proper indentation बहुत जरूरी है।

3. Examples

	Example Code	Explanation	Output
①	<pre>x = 10 if x > 5: print("x is greater than 5") print("End of program")</pre>	x की value 10 है और condition True है, इसलिए if block execute होगा।	x is greater than 5 End of program
②	<pre>x = 3 if x > 5: print("x is greater than 5") print("End of program")</pre>	x की value 3 है और condition False है, इसलिए if block skip होगा।	End of program

4. Indentation Example (Very Important)

<u>Correct Indentation</u> (✓)	<u>Incorrect Indentation</u> (X)
<pre>age = 18 if age >= 18: print("You are eligible to vote.") print("Thank You!") (Proper indentation)</pre>	<pre>age = 18 if age >= 18: print("You are eligible to vote.") print("Thank You!") (Improper indentation) → IndentationError</pre>

Note :

Python में code की readability और structure indentation पर depend करता है।

5. Points to Remember

- if statement अकेले भी use किया जा सकता है।
- condition के बाद colon (:) लगाना अनिवार्य है।
- if block के सभी statements का indentation समान होना चाहिए।
- if statement case sensitive होता है (True और true अलग होते हैं)।

Use :

if statement का use उन situations में होता है जहाँ हमें किसी condition के True होने पर कुछ action करना होता है।

Topic : Relational and Logical Operators

- Python में operators का उपयोग values और expressions को evaluate करने के लिए किया जाता है।
- Relational Operators दो values के बीच relation check करते हैं और result True या False return करते हैं।
- Logical Operators conditions को combine करते हैं।

1. Relational Operators

- Relational Operators का उपयोग दो values के बीच comparison करने के लिए होता है।
- Result हमेशा Boolean (True या False) मिलता है।

Relational Operators Example

$10 > 5 \rightarrow \text{True}$
 $10 == 5 \rightarrow \text{False}$
 $10 != 5 \rightarrow \text{True}$
 $5 <= 5 \rightarrow \text{True}$

<u>Operator</u>	<u>Name</u>	<u>Meaning</u>	<u>Example</u>	<u>Result</u>
$==$	Equal to	दोनों values बराबर हैं।	$5 == 5$	True
$!=$	Not equal to	दोनों values बराबर नहीं हैं।	$5 != 3$	True
$>$	Greater than	बाएँ value दाएँ value से बड़ी है।	$7 > 4$	True
$<$	Less than	बाएँ value दाएँ value से छोटी है।	$3 < 6$	True
$>=$	Greater than or equal to	बाएँ value बड़ी या बराबर है (right value से)	$6 >= 6$	True
$<=$	Less than or equal to	बाएँ value छोटी या बराबर है (right value से)	$4 <= 9$	True

2. Logical Operators

- Logical Operators multiple conditions को combine करने के लिए उपयोग होते हैं।
- Result हमेशा True या False होता है।
- Python में तीन Logical Operators होते हैं: and, or, not.

(A) and Operator

- and operator तब True return करता है जब दोनों conditions True हों।
- किसी भी एक condition False होने पर result False होता है।

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

Example :

$x = 10, y = 5$

$(x > 5) \text{ and } (y < 10) \rightarrow \text{True and True} \rightarrow \text{True}$

$(x < 5) \text{ and } (y > 10) \rightarrow \text{False and False} \rightarrow \text{False}$

(B) or Operator

- or operator तब True return करता है जब कम से कम एक condition True हों।
- दोनों conditions False होने पर ही result False होता है।

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

Example :

$(x > 5) \text{ or } (y > 10) \rightarrow \text{True or False} \rightarrow \text{True}$

$(x < 5) \text{ or } (y < 3) \rightarrow \text{False or False} \rightarrow \text{False}$

(C) not Operator

- not operator एक ही condition को reverse (उल्टा) कर देता है।
- True को False और False को True कर देता है।

A	<u>not A</u>
True	False
False	True

Example :

$x = 10$

$\text{not } (x > 5) \rightarrow \text{not True} \rightarrow \text{False}$

$\text{not } (x < 5) \rightarrow \text{not False} \rightarrow \text{True}$

Key Points :

- * Relational operators comparison के लिए उपयोग होते हैं।
- * Logical operators multiple conditions को combine करते हैं।
- * Result हमेशा Boolean (True/False) होता है।

Topic : Bit Wise Operators

- Bitwise Operators bits पर काम करते हैं, न कि पूरे numbers पर।
- ये operators integers के binary representation पर perform करते हैं।
- इनका उपयोग low level programming, system programming और performance critical applications के लिए किया जाता है।
- Python में Bitwise Operators निम्नलिखित हैं: $\&$, $|$, $\wedge$, $\sim$, $\ll$, $\gg$

1. Bitwise Operators की सूची

Operator	Name	Meaning	Example	Result
$\&$	AND	दोनों bits 1 होने पर 1	$5 \& 3$	1
$ $	OR	दोनों bits में से कोई एक 1 हो तो 1	$5 3$	7
$\wedge$	XOR	दोनों bits अलग हैं तो 1	$5 \wedge 3$	6
$\sim$	NOT	सभी bits को उल्टा करता है (1's complement)	~ 5	-6
$\ll$	Left Shift	Bits को बायीं ओर shift करता है	$5 \ll 1$	10
$\gg$	Right Shift	Bits को दायीं ओर shift करता है	$5 \gg 1$	2

2. Bitwise Operators कैसे काम करते हैं? (उदाहरण सहित)

(A) AND Operator ($\&$)

उदाहरण : $A = 5$, $B = 3$

Binary में,

$A = 5 \rightarrow 0000\ 0101$

$B = 3 \rightarrow 0000\ 0011$

$A \& B \rightarrow 0000\ 0001 \rightarrow 1$

Rule :

$1 \& 1 = 1$
बाकी सभी स्थिति में 0

(B) OR Operator ($|$)

$A = 5 \rightarrow 0000\ 0101$

$B = 3 \rightarrow 0000\ 0011$

$A | B \rightarrow 0000\ 0111 \rightarrow 7$

Rule :

$1 | 1 = 1$
 $1 | 0 = 1$
 $0 | 1 = 1$
 $0 | 0 = 0$

(C) XOR Operator ($\wedge$)

$A = 5 \rightarrow 0000\ 0101$

$B = 3 \rightarrow 0000\ 0011$

$A \wedge B \rightarrow 0000\ 0110 \rightarrow 6$

Rule :

$1 \wedge 1 = 0$
 $1 \wedge 0 = 1$
 $0 \wedge 1 = 1$
 $0 \wedge 0 = 0$

(D) NOT Operator ($\sim$)

$A = 5 \rightarrow 0000\ 0101$

$\sim A \rightarrow 1111\ 1010$

(In Decimal) $\rightarrow -6$

Python में $\sim A = -(A+1)$ होता है।

(E) Left Shift ($\ll$)

$A = 5 \rightarrow 0000\ 0101$

$A \ll 1 \rightarrow 0000\ 1010 \rightarrow 10$

$A \ll 2 \rightarrow 0001\ 0100 \rightarrow 20$

Left shift में दाएँ ओर 0 fill होते हैं।

(F) Right Shift ($\gg$)

$A = 5 \rightarrow 0000\ 0101$

$A \gg 1 \rightarrow 0000\ 0010 \rightarrow 2$

$A \gg 2 \rightarrow 0000\ 0001 \rightarrow 1$

Right shift में बाएँ ओर sign bit (0) fill होता है।

3. Example (सभी एक साथ)

$a = 5$, $b = 3$

$a \& b = 1$

$a | b = 7$

$a \wedge b = 6$

$\sim a = -6$

$a \ll 1 = 10$

$a \gg 1 = 2$

Note :

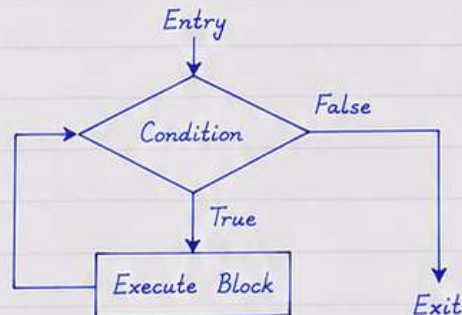
Bitwise Operators का उपयोग कभी-कभी memory management, encryption, graphics, और low level programs में किया जाता है।

Topic : The while Loop

- while loop Python में एक entry controlled loop होता है।
- यह loop condition के True होने तक बार-बार execute होता है।
- जैसे ही condition False होती है, loop terminate हो जाता है और control loop के बाहर चला जाता है।
- while loop उन situations में use किया जाता है जहाँ हमें नहीं पता होता कि loop कितनी बार चलेगा।

1. Syntax

```
while condition :  
    statement(s)  
    (update statement)
```



2. How it Works ?

- सबसे पहले condition check होती है।
- अगर condition True है, तो loop के अंदर का block execute होता है।
- फिर update statement के द्वारा condition को change किया जाता है।
- इसके बाद दोबारा condition check होती है।
- यदि condition False हो जाती है, तो loop stop हो जाता है और control loop के बाहर चला जाता है।

Important :

- while loop में कम-से-कम condition एक बार check जरूर होती है।
- infinite loop से बचने के लिए update statement जरूरी है, जो condition को False कर सके।

3. Flow of while Loop

Start → Check Condition → (True) Execute Block → Update Condition → Check Condition

(False) ↓
Exit

4. Example Program

1 से 5 तक numbers print करना

```
i = 1  
while i <= 5 :  
    print(i)  
    i = i + 1
```

Output :

1
2
3
4
5

Example : Sum of first n natural numbers

```
n = 5  
i = 1  
sum = 0  
while i <= n :  
    sum = sum + i  
    i = i + 1  
print("Sum =", sum)
```

Output :

Sum = 15

5. Infinite Loop (सावधानी).

```
i = 1  
while i <= 5 :  
    print(i)  
    # i = i + 1 (अगर यह line  
    नहीं लिखी तो loop कभी  
    खत्म नहीं होगा)
```

Result :

यह loop
infinite
रूप से
चलता रहेगा।

6. Uses of while Loop

- जब हमें पहले से नहीं पता कि loop कितनी बार चलेगा।
- Menu driven programs में।
- User input validate करने के लिए।
- File processing, game loops, data reading आदि में।

Note :

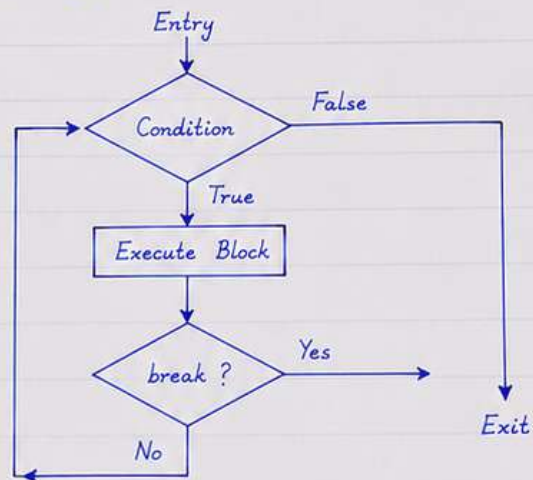
- while loop एक powerful tool है repeated tasks करने के लिए।
- हमेशा condition और update statement को सही तरह से design करें।
- गलत use करने पर infinite loop की संभावना रहती है।

Topic : break and continue

- Python में break और continue loop control statements होते हैं।
- इनका उपयोग loop के execution को control करने के लिए किया जाता है।
- break loop को terminate करता है जबकि continue current iteration को skip करके next iteration पर चला जाता है।

1. break Statement

- break statement का उपयोग loop को तुरंत terminate करने के लिए होता है।
- जब break execute होता है, तब loop का control loop के बाहर चला जाता है।
- break के बाद loop के अंदर का कोई भी code execute नहीं होता।



Example : Print numbers from 1 to 10,
but break the loop when $i == 5$

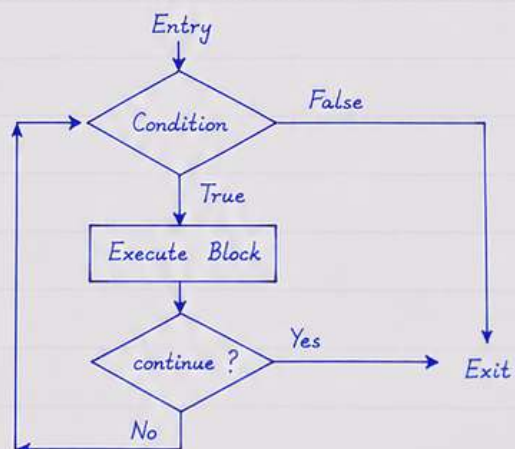
```
for i in range(1, 11):  
    if i == 5:  
        break  
    print(i)
```

Output :

1
2
3
4

2. continue Statement

- continue statement का उपयोग current iteration को skip करने के लिए होता है।
- जब continue execute होता है, तब loop का control अगले iteration पर चला जाता है।
- continue के बाद loop के शेष statements उस iteration के लिए execute नहीं होते।



Example : Print numbers from 1 to 10,
but skip number 5

```
for i in range(1, 11):  
    if i == 5:  
        continue  
    print(i)
```

Output :

1
2
3
4
6
7
8
9
10

Key Points :

- * break loop को पूरी तरह terminate करता है।
- * continue केवल current iteration को skip करता है।
- * break और continue केवल loop के अंदर ही use होते हैं (for, while)।

break vs continue		
Point	break	continue
कार्य	loop को terminate करता है	current iteration को skip करता है
Control कहाँ जाता है ?	loop के बाहर	loop के अगले iteration पर
आगे का code	loop के बाद का code execute होगा	उसी loop का अगला iteration execute होगा
उदाहरण	search में item मिलने पर loop रोकना	odd numbers को skip करके even print करना

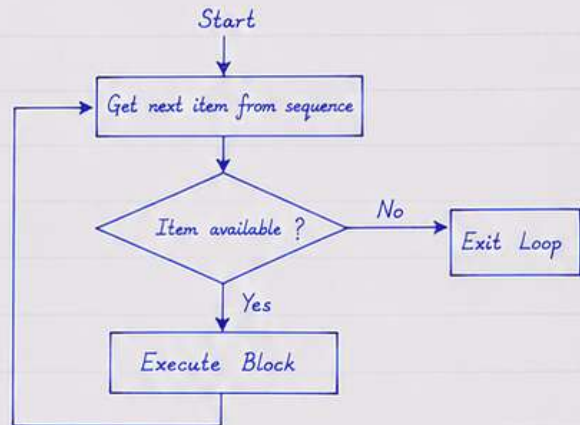
Topic : The for Loop

- for loop Python में एक controlled loop होता है जो किसी sequence (जैसे list, tuple, string, range आदि) के items पर iterate करता है।
- यह loop निश्चित बार चलता है, क्योंकि हमें पता होता है कि items कितने हैं।
- for loop का उपयोग तब करते हैं, जब हमें किसी sequence के प्रत्येक element पर कुछ operation करना हो।

1. Syntax

```
for variable in sequence :  
    statement(s)  
    (indented block)
```

- variable → हर iteration में next item को hold करता है।
- sequence → वह collection जिसके items पर loop चलता है।
- statement(s) → loop के अंदर execute होने वाले statements।



2. How it Works ?

- for loop sequence के पहले item से शुरू होता है।
- variable उस item को लेता है और block execute होता है।
- फिर sequence का अगला item लिया जाता है और process दोहराया जाता है।
- जब sequence में कोई item नहीं बचता, loop terminate हो जाता है।

Important :

for loop में हमें initialization, condition check और update अलग से लिखने की जरूरत नहीं होती, यह सब Python automatic handle करता है।

3. Example Programs

(A) 1 से 5 तक numbers print करना

```
for i in range(1, 6):  
    print(i)
```

Output :

1
2
3
4
5

(B) List के सभी items print करना

```
fruits = ['apple', 'mango', 'banana']  
for fruit in fruits:  
    print(fruit)
```

Output :

apple
mango
banana

(C) String के characters print करना

```
for ch in "Hello":  
    print(ch)
```

Output :

H
e
l
l
o

(D) 1 से 10 तक even numbers का sum

```
s = 0  
for i in range(2, 11, 2): # step = 2  
    s = s + i  
print("Sum =", s)
```

Output :

Sum = 30

4. range() Function

- range() for loop के साथ अक्सर use होता है।
- range(start, stop, step)
- start → default 0
- stop → इस number तक, लेकिन यह include नहीं होता।
- step → increment (default 1)

Examples :

- range(5) → 0, 1, 2, 3, 4
- range(1, 6) → 1, 2, 3, 4, 5
- range(2, 10, 2) → 2, 4, 6, 8

5. Key Points

- for loop known number of times iterate करता है।
- यह sequence के हर item को एक बार process करता है।
- for loop को list, tuple, string, range, set, dict आदि पर use किया जा सकता है।
- loop के अंदर break और continue का use किया जा सकता है।

Note : • for loop लिखना आसान और code को clean बनाता है, इसलिए इसे अधिकतर situations में prefer किया जाता है।